AI: Homework 4 Report

Sri Bhuvana Vaishnavi Dasika - sd3971

Part I: Classification

The dataset (input.csv) contains 500 two-dimensional points labelled with two classes that form a chessboard-like pattern. Roughly square regions of class 0 and class 1 alternate across the [0, 4] x [0, 4] plane, which makes the boundary highly non-linear.

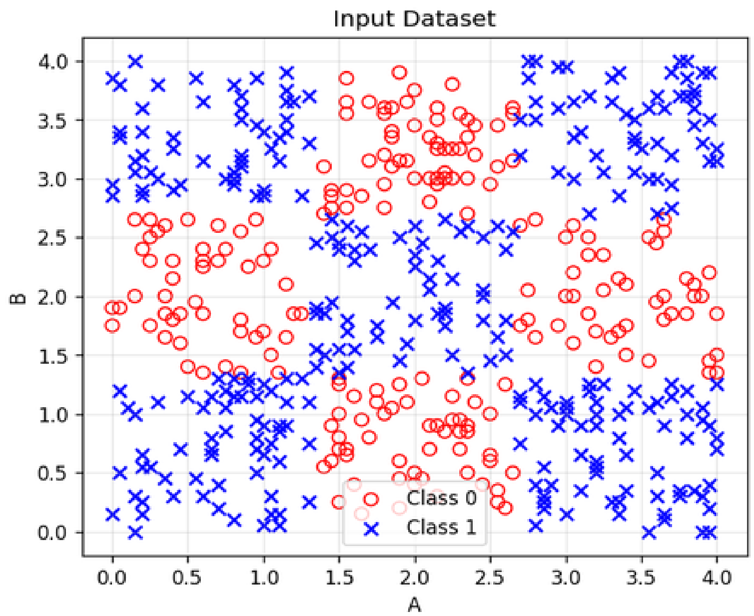

Figure 1: Scatter plot of the input dataset. Class 0 (red circles) and Class 1 (blue x markers) alternate in a chessboard pattern.

Methodology

The data was split into 60% training and 40% testing using sklearn's ***train_test_split*** with stratification (random_state=42). For each of the five classifiers, hyperparameters were tuned with ***GridSearchCV*** using 5-fold cross-validation on the training set. The best CV score and the held-out test accuracy were recorded, and the decision boundary of each best model was plotted on the test set.

Results

Classifier	Best CV Train Acc.	Test Accuracy
k-Nearest Neighbors	0.9467	0.9400
Logistic Regression	0.5900	0.5900
Decision Tree	0.9800	0.9950
Random Forest	0.9467	0.9800
AdaBoost	0.6833	0.6250

Decision Boundaries

The figures below show the test set with each model's predicted decision regions in the background (red = class 0, blue = class 1).

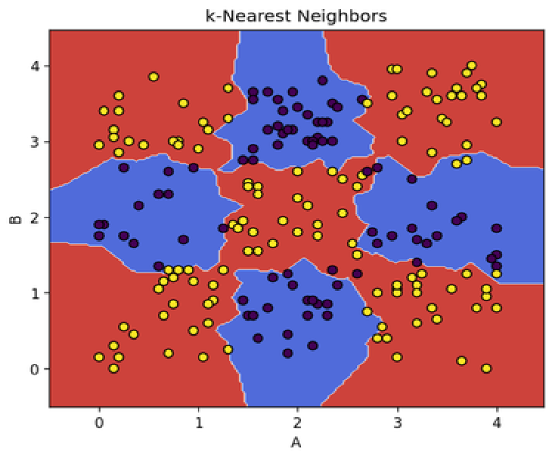

Figure 2: k-NN (k=1, leaf_size=5).

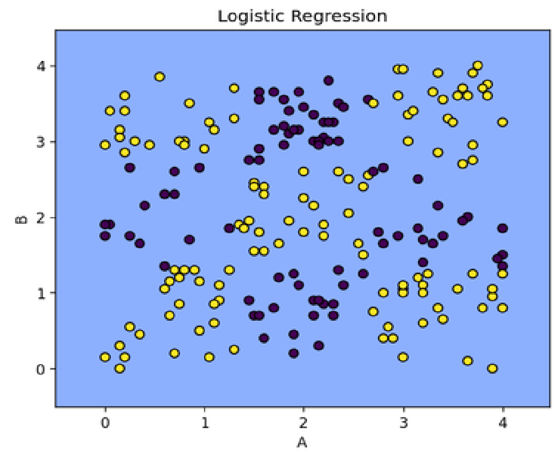

Figure 3: Logistic Regression (C=0.1).

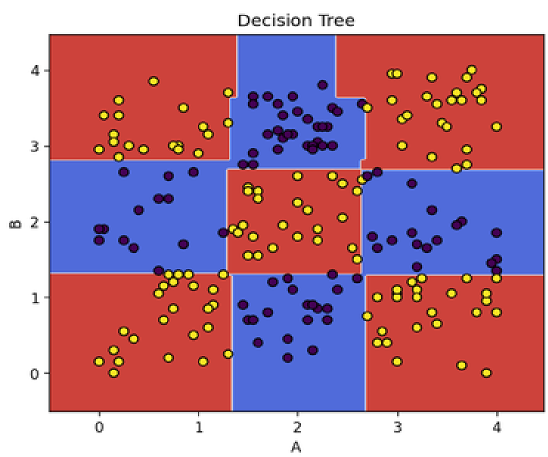

Figure 4: Decision Tree (max_depth=8, min_samples_split=2).

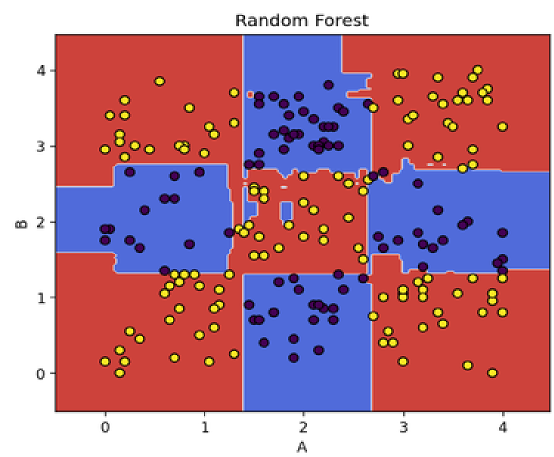

Figure 5: Random Forest (max_depth=5, min_samples_split=2).

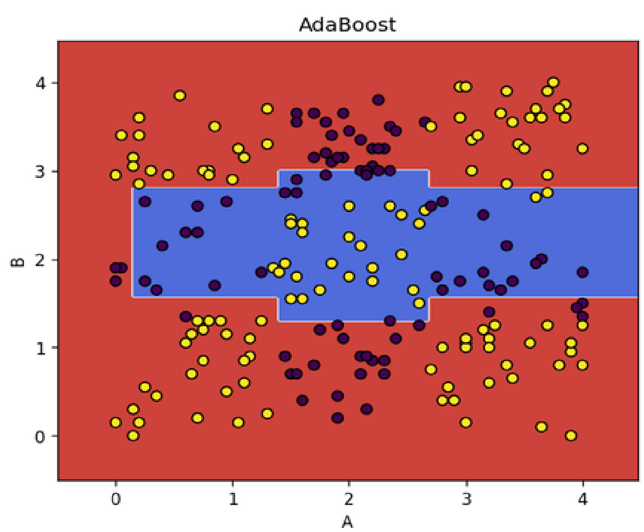

Figure 6: AdaBoost (n_estimators=50).

Discussion of Performance

Decision Tree was the clear winner, achieving 0.98 cross-validation accuracy and 0.995 on the test set. This is unsurprising: the chessboard structure is built from ***axis-aligned splits***, which is exactly what a CART tree produces. With max_depth=8, the tree has enough capacity to carve out every square block, and the resulting decision boundary in Figure 4 looks almost identical to the true generative pattern.

Random Forest came in second (0.95 CV / 0.98 test). The constraint ***max_depth* ≤ *5*** in the grid limits each tree's capacity, so the ensemble has to compensate through averaging. The boundary in Figure 5 is still chessboard-like, but with slightly fuzzier and less crisp edges than the single deep tree.

k-Nearest Neighbors with k=1 also performed very well (0.95 CV / 0.94 test). KNN is well-suited to this dataset because nearby points almost always share a class within each chessboard square; however, the boundaries (Figure 2) are jagged and follow the local arrangement of training points rather than the underlying square pattern, which makes it slightly less robust than the tree.

AdaBoost with the default 1-level decision stumps did poorly (0.68 CV / 0.625 test). A single stump can only make one axis-aligned cut, and AdaBoost's weighted majority vote of stumps cannot easily represent a multi-cell chessboard with limited estimators. The boundary in Figure 6 picks up only a horizontal band rather than the full pattern.

Logistic Regression performed essentially at chance (0.59 / 0.59). It can only fit a single straight-line boundary, and no line separates the two classes in a chessboard layout. The decision region in Figure 3 collapses to a single colour, predicting one class everywhere.

Conclusion. Decision Tree is the best method for this dataset, with Random Forest and k-NN as strong runners-up. The takeaway is that model choice should match the geometry of the problem: linear models fail dramatically on non-linearly separable data, while tree-based methods exploit the axis-aligned structure of the chessboard naturally.

Part II: High Dimensionality : Curse or Blessing?

Experimental Plot

We generate Gaussian datasets of varying dimensionality (d = 1 to 100) and four sample sizes (N = 100, 500, 1 000, 5 000). For each (N, d) pair, a random query point is drawn from the data, and we compute the ratio of the maximum to the minimum Euclidean distance from the query to all other points.

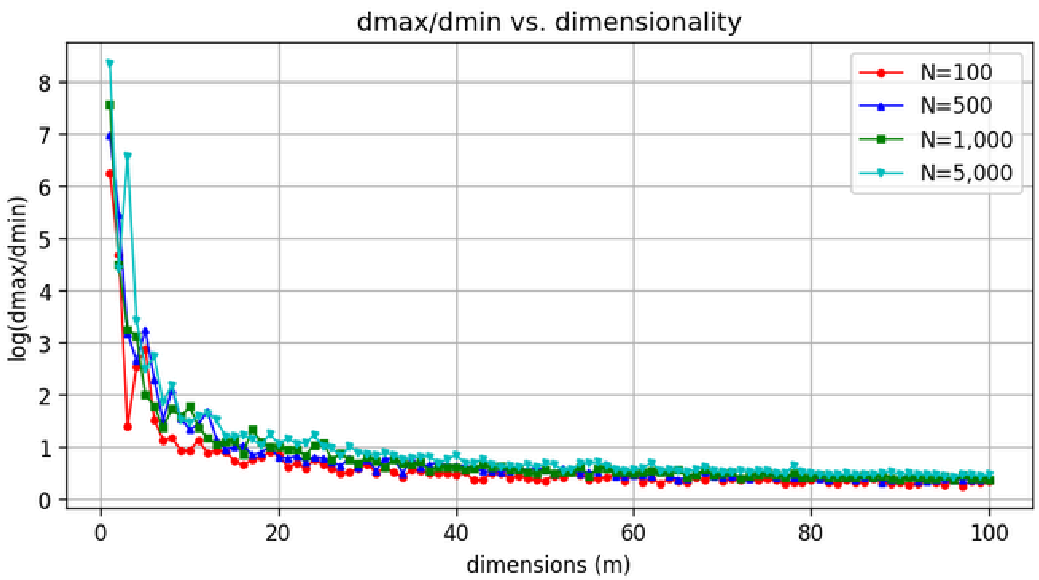

Figure 7: log(dmax/dmin) versus dimensionality for several sample sizes. The ratio collapses rapidly toward 1 (i.e. log toward 0).

Findings. All four curves drop sharply as ***d*** grows. By d ~ 50 the log-ratio is well below 1 (so dmax/dmin < 3), and by d = 100 it sits near 0.4 (dmax/dmin ~ 1.5). In other words, the farthest and nearest points become almost equidistant from the query. This is the canonical statement of the curse of dimensionality: ***distance loses its discriminative power in high dimensions***, which directly breaks the central assumption of distance-based methods such as k-NN. The effect is essentially independent of the sample size N, consistent with the theoretical results of Beyer et al. (1999).

The Blessing of High Dimensionality

The phrase “curse of dimensionality,” coined by Bellman (1961), captures several well-known pathologies: distances concentrate, data becomes exponentially sparse, and statistical estimation suffers from the bias–variance tradeoff. The plot above demonstrates one of these pathologies cleanly. Yet a parallel research literature, developed largely after 2000, argues that high dimensionality is often a ***blessing*** rather than a curse, particularly for modern machine learning. Three threads of this argument are worth highlighting.

1. Concentration of measure. Donoho (2000) observes that in high-dimensional spaces, smooth functions of many independent variables concentrate sharply around their expectations. This is the same phenomenon that powers the law of large numbers: averages become more predictable as more random variables are summed. In machine learning this translates to surprisingly stable behavior of high-dimensional estimators, for example, regularized risk minimizers and kernel methods exhibit tighter generalization bounds in higher dimensions when the underlying signal is structured (Gorban et al., 2018).

2. Linear separability becomes generic. Cover's theorem (1965) shows that random points in sufficiently high-dimensional spaces are almost always linearly separable. Gorban & Tyukin (2018) extend this with their ***stochastic separation theorems***: in high dimensions, any single point can be separated from a large random sample by a simple linear classifier with probability tending to 1. This is one of the reasons why neural networks succeed when we lift raw inputs into very high-dimensional internal representations. The classes that were tangled in the input space become cleanly separable in feature space. Modern over-parameterized networks lean on exactly this geometric fact.

3. Real data lives on low-dimensional manifolds. Domingos (2012) in “A Few Useful Things to Know about Machine Learning” stresses that the curse of dimensionality is mitigated by the ***blessing of non-uniformity***: although the ambient space is high-dimensional, real-world data tends to concentrate on a much lower-dimensional manifold. Images, text embeddings, and gene expression data are all examples. Manifold learning, autoencoders, and modern self-supervised representation learning explicitly exploit this. The informative variance is far smaller than the apparent dimensionality would suggest, and many learning algorithms behave as if the ***intrinsic*** dimension were what matters, not the ambient one.

My view. The curse and the blessing are not contradictions but describe different regimes. When data is genuinely uniform in a high-dimensional Euclidean ball. As in our synthetic experiment, distances really do flatten and naive distance-based methods fail. But practical data is highly structured: there is signal, redundancy, and manifold geometry. Methods that respect that structure (linear separators in lifted spaces, deep representations, manifold-aware regularizers) can convert raw dimensionality into an asset. In other words, the curse is a property of the geometry of uniform noise; the blessing is a property of how learning algorithms ***interact*** with structured high-dimensional signals.

References

- Bellman, R. E. (1961). ***Adaptive Control Processes: A Guided Tour***. Princeton University Press.
- Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When is “Nearest Neighbor” Meaningful? ***ICDT 1999***, pp. 217–235.
- Cover, T. M. (1965). Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition. ***IEEE Trans. Electronic Computers***, EC-14(3), 326–334.
- Domingos, P. (2012). A Few Useful Things to Know about Machine Learning. ***Communications of the ACM***, 55(10), 78–87.
- Donoho, D. L. (2000). High-Dimensional Data Analysis: The Curses and Blessings of Dimensionality. ***AMS Math Challenges Lecture***.
- Gorban, A. N., & Tyukin, I. Y. (2018). Blessing of Dimensionality: Mathematical Foundations of the Statistical Physics of Data. ***Phil. Trans. Royal Society A***, 376:20170237.
- Wikipedia contributors. ***Curse of Dimensionality***. Retrieved from https://en.wikipedia.org/wiki/Curse_of_dimensionality.